МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность АС-68

Выполнил
Борисюк
Р.Б.
студент группы АС-68

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Вариант 1

• *Iris*
• Определить вид ириса (setosa, versicolor, virginica) по измерениям его цветка
• Задания:

1. Загрузите данные и ознакомьтесь с ними;
2. Разделите выборку на обучающую и тестовую;
3. Обучите модели k-NN, Decision Tree и SVM. Для k-NN попробуйте найти оптимальное значение *k*;
4. Оцените точность (accuracy) каждой модели на тестовой выборке;
5. Сравните результаты и сделайте вывод, какая модель лучше всего справилась с этой задачей.

Выполнение:

Код программы

```
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

iris = load_iris()

X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = iris.target

print("ДАТАСЕТ IRIS")
print("Количество объектов:", len(X))
print("Количество признаков:", len(X.columns))
print("Количество классов:", len(iris.target_names))
print("Классы:", iris.target_names)

print("\nПервые 5 строк:")
print(X.head())

print("\nРаспределение классов:")
print(pd.Series(y).value_counts().sort_index())

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

```
)

print("\nРазмер обучающей выборки:", len(X_train))
print("Размер тестовой выборки:", len(X_test))

k_values = range(1, 16)
knn_scores = []

print("\nПОДБОР ОПТИМАЛЬНОГО K")

for k in k_values:
    model = make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=k)
    )

    scores = cross_val_score(
        model, X_train, y_train, cv=5, scoring="accuracy"
    )

    knn_scores.append(scores.mean())
    print("K =", k, "| Средняя точность:", round(scores.mean(), 4))

best_k = list(k_values)[knn_scores.index(max(knn_scores))]

print("\nОптимальное значение K:", best_k)

knn = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=best_k)
)

tree = DecisionTreeClassifier(random_state=42)

svm = make_pipeline(
    StandardScaler(),
    SVC(kernel="rbf", random_state=42)
)

models = {
    "k-NN": knn,
    "Decision Tree": tree,
    "SVM": svm
}

results = {}

print("\nРЕЗУЛЬТАТЫ КЛАССИФИКАЦИИ")

for name, model in models.items():
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    accuracy = accuracy_score(y_test, predictions)
```

```
    results[name] = accuracy

    print(name, "| Accuracy:", round(accuracy, 4))

print("\nСРАВНЕНИЕ МОДЕЛЕЙ")

for name, accuracy in results.items():
    print(name, "-", round(accuracy * 100, 2), "%")

best_model = max(results, key=results.get)

print("\nЛУЧШАЯ МОДЕЛЬ:", best_model)
print("ТОЧНОСТЬ:", round(results[best_model] * 100, 2), "%")
```

Результат:

```
ДАТАСЕТ IRIS
Количество объектов: 150
Количество признаков: 4
Количество классов: 3
Классы: ['setosa' 'versicolor' 'virginica']

Первые 5 строк:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

Распределение классов:
0    50
1    50
2    50
Name: count, dtype: int64

Размер обучающей выборки: 120
Размер тестовой выборки: 30

ПОДБОР ОПТИМАЛЬНОГО K
K = 1 | Средняя точность: 0.9417
K = 2 | Средняя точность: 0.95
K = 3 | Средняя точность: 0.9583
K = 4 | Средняя точность: 0.9583
K = 5 | Средняя точность: 0.9667
K = 6 | Средняя точность: 0.9583
K = 7 | Средняя точность: 0.9583
K = 8 | Средняя точность: 0.9583
K = 9 | Средняя точность: 0.9583
K = 10 | Средняя точность: 0.9667
K = 11 | Средняя точность: 0.9583
K = 12 | Средняя точность: 0.9667
K = 13 | Средняя точность: 0.95
K = 14 | Средняя точность: 0.9583
K = 15 | Средняя точность: 0.9583

Оптимальное значение K: 5

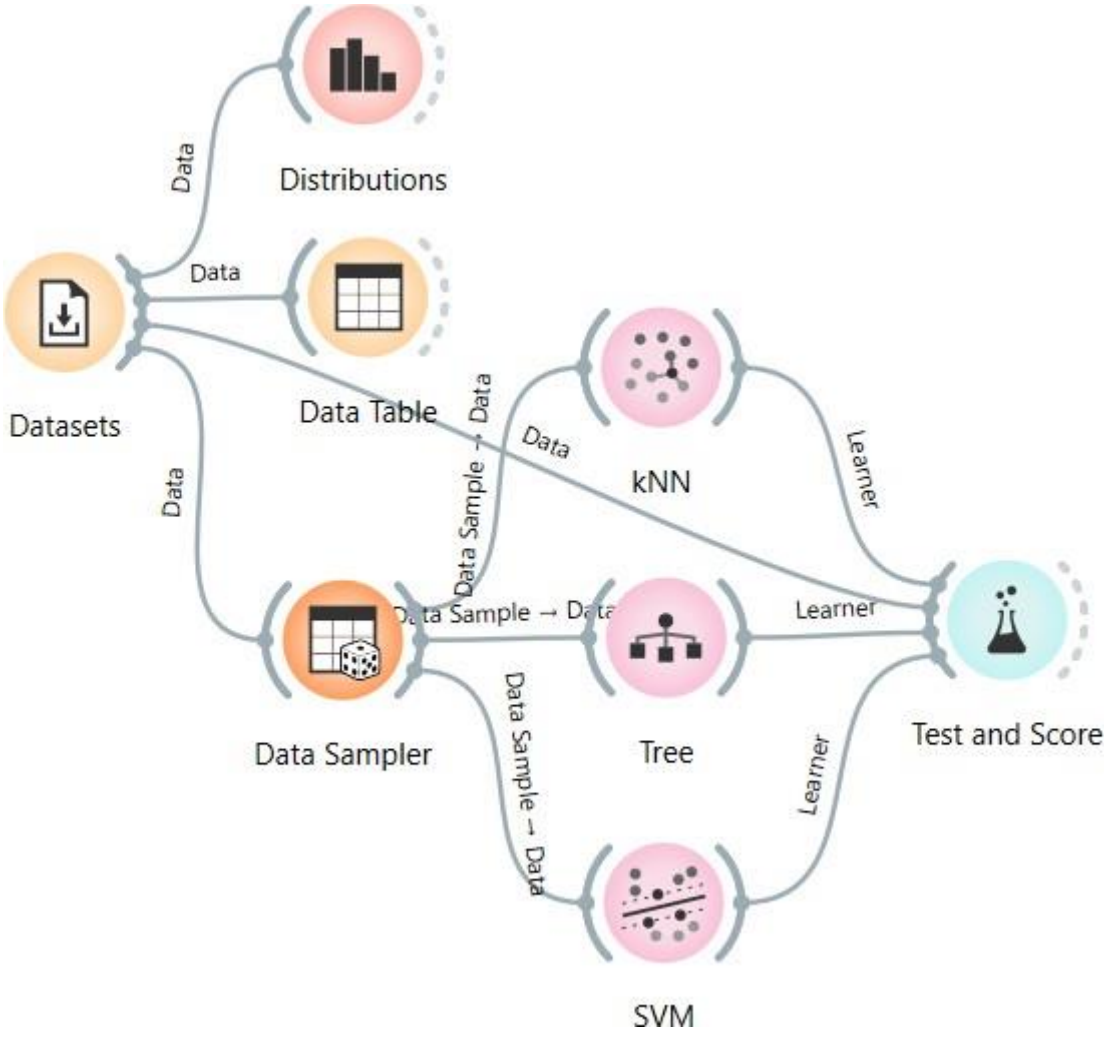
РЕЗУЛЬТАТЫ КЛАССИФИКАЦИИ
k-NN | Accuracy: 0.9333
Decision Tree | Accuracy: 0.9333
```

SVM | Accuracy: 0.9667

СРАВНЕНИЕ МОДЕЛЕЙ
k-NN - 93.33 %
Decision Tree - 93.33 %
SVM - 96.67 %

ЛУЧШАЯ МОДЕЛЬ: SVM
ТОЧНОСТЬ: 96.67 %

Результаты в Orange:



Model	AUC	CA	F1	Prec	Recall	MCC
Tree	0.974	0.947	0.947	0.947	0.947	0.920
kNN	0.987	0.967	0.967	0.967	0.967	0.950
SVM	0.997	0.960	0.960	0.960	0.960	0.940

Вывод: В ходе лабораторной работы выполнена классификация вина с помощью k-NN, дерева решений и SVM. Наилучший показатель качества продемонстрировал алгоритм k-NN, достигнув значения F1-score = 0,967, а наилучшую эффективность при программной реализации на Python продемонстрировал метод опорных векторов (SVM), достигнув точности (Accuracy) 96,67% на тестовой выборке.